CS 166 ASSIGNMENT 5 SOLUTION

Overview

Marks on question:

- No mark: 1-part short answer question (the key point(s) for explanation is underlined)
- !: purely auto-graded question (see quiz answers)
- +: multi-part question: the key point(s) for explanation is underlined.

Grading rules:

- Those 0.5 pt sub-questions are all-or-nothing.
- For 1 pt yes/no question, wrong answer or answer without explanation will result in a 0.
- For whether message proves identity or not...
  - If the answer is yes, 0.5 pt for explaining why only XXX can get the correct response and 0.5 pt for explaining how the other party can verify the response.
  - If the answer is no, partial credits (0.5) if is on the right track but didn't include all key points.

• You will get 0.5 (on the whole question, not each sub-question) as long as you wrote something meaningful.

Week 9 – Conceptual Protocols (Q1 – Q10)

!Q1: : Match the term with the descriptions. (L19 P7 – 9 & L20 P15)

See quiz answers.

!Q2: What is the purpose of Perfect Forward Secrecy? (L20 P14)

See quiz answers.

Q3: Why sending the name(s)? (L19 & 20)

Since the other party needs to know which shared key or whose public key to use for decryption/encryption.

Q4: Using timestamp pros & cons (L20 P11)

I. Pros: T is public so no need to send a separate message for nonce, which can reduce the number of messages needed.

II. Cons: time skew is needed for syncing/delays, which increases the chance of replay attack.

(Next question starts on the next page)

Yan Chen
Computer Science Department

+Q5: Authentication by symmetric key with nonces – 1 (L19 P11 - 12)

```
Protocol:
1) Alice -> Bob: "I'm Alice", R
2) Bob -> Alice: E(R, KAB)
3) Alice -> Bob: E(R + 1, KAB)
```

5.1) If not consider any attacks...

I. No, message 1 doesn't show Alice is Alice. R is just a random number, so anyone can send it.

II. Yes, message 3 shows Alice is Alice. Since K_{AB} is known only to Alice & Bob, being able to encrypt R + 1 using K_{AB} shows Alice is Alice and Bob got R from the 1st message to verify if E(R + 1, K_{AB}) is correct.

III. Yes, message 2 shows Bob is Bob. Since K_{AB} is known only to Alice & Bob, being able to encrypt R using K_{AB} shows Bob is Bob and Alice knows R to verify if E(R, K_{AB}) is correct.

5.2) Consider replay attack...

Yes, Trudy can do a reflection attack since the protocol asks Alice & Bob to prove identity in the "same" way – encrypting a message using K_{AB}. That is, Trudy can get the 3rd message by opening a 2nd session and sending R + 1 to Bob as "Alice", then Bob will reply her E(R + 1, K_{AB}), which is what Trudy needs.

+Q6: Authentication by symmetric key with nonces - 2 (L19 P11 - 13)

```
Protocol:
1) Alice -> Bob: "I'm Alice", E(R, KAB)
2) Bob -> Alice: R + 1
3) Alice -> Bob: E(R + 1, KAB)
```

6.1) If not consider any attacks...

I. No, message 1 doesn't show Alice is Alice. Bob didn't know R yet so he can't verify if E(R, K_{AB}) is correct.

II. Yes, message 3 shows Alice is Alice. Since K_{AB} is known only to Alice & Bob, being able to encrypt R + 1 using K_{AB} shows Alice is Alice and Bob now knows R to verify if E(R + 1, K_{AB}) is correct.

III. Yes, message 2 shows Bob is Bob. Since K_{AB} is known only to Alice & Bob, being able to decrypt E(R, K_{AB}) to get R shows Bob is Bob and Alice knows R to verify if R + 1 is correct.

6.2) No, a reflection attack is not possible. Alice & Bob are doing "different" things to prove their identities: Bob proves his identity by decrypting a message while Alice proves her identity by encrypting a message.

(Next question starts on the next page)

+Q7: Authentication by public key with nonces (L20 P6 – 9)

```
Protocol:
1) Alice -> Bob: "I'm Alice", {K}Bob, [RA]Alice
2) Bob -> Alice: {K, RB}Alice
3) Alice -> Bob: RB
```

7.1) If NOT consider any attacks...

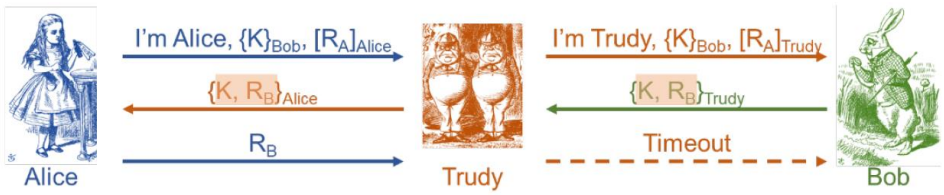
I. No, message 1 doesn't show Alice is Alice. Everyone can compute $\{K\}_{Bob}$ using Bob's public key, and Bob cannot verify Alice's signature on R_A since he didn't know the correct value of R_A.

II. Yes, message 3 shows Alice is Alice. R_B is encrypted using Alice's public key so only Alice can decrypt $\{K, R_B\}_{Alice}$ with her private key to get R_B. And Bob knows R_B to verify if it is correct.

III. Yes, message 2 shows Bob is Bob. K is encrypted using Bob's public key so only Bob can decrypt $\{K\}_{Bob}$ by his private key to get K. And Alice knows K to verify if it is correct after decrypting $\{K, R_B\}_{Alice}$.

7.2) Consider attacks...

No, K is not exchanged securely since MiM attack is possible. Trudy can be in the middle forwarding the message from Alice to Bob but as Trudy, then Bob will send her $\{K, R_B\}_{Trudy}$ which Trudy can decrypt to get K, R_B (i.e., K is exposed!). Finally, Trudy can also compute $\{K, R_B\}_{Alice}$ since it's using Alice's public key, which means she can pretend to be Bob.



7.3) Change message 2 from $\{K, R_B\}_{Alice}$ to $\{K\}_{Alice}$, $[R_B]_{Bob}$...

Message 2 still shows Bob is Bob for the same reason as 7.1 – III.

However, the answer to 7.1 – II changes: Message 3 doesn't show Alice is Alice in this case since everyone can decrypt $[R_B]_{Bob}$ to get R_B using Bob's public key.

+Q8: Authentication by public key with timestamp (L20 P12 – 13)

```
Protocol:
1) Alice -> Bob: {"I'm Alice", [T, K]Alice}Bob
2) Bob -> Alice: [T + 1]Bob
```

8.1) If NOT consider any attacks...

I. Yes, message 1 shows Alice is Alice. Only Alice can compute $[T, K]_{Alice}$ using her private key and Bob knows T to verify the Alice's signature.

II. Yes, message 2 shows Bob is Bob. Only Bob can compute $[T + 1]_{Bob}$ using his private key and Alice knows T + 1 to verify Bob's signature.

III. Yes, K is exchanged securely. K only appeared in the 1st message which is encrypted using Bob's public key, so only Bob can decrypt it to get K using his private key. And since the 2nd message doesn't include K, Trudy can't get it by opening another session.

8.2) Change message 2 from {"I'm Alice", $[T, K]_{Alice}\}_{Bob}$ to {"I'm Alice", T, $[K]_{Alice}\}_{Bob}$...

Message 2 still shows Bob is Bob for the same reason as 8.1 – II.

However, the answer to 8.1 – I changes: Message 1 doesn't show Alice is Alice in this case since Bob doesn't know K to verify Alice's signature.

+Q9: Authentication by public key with timestamp and Diffie-Hellman (L20 P12 – 15)

Protocol:
1) Alice -> Bob: "I'm Alice", $[T]_{Alice}$, g^a mod p
2) Bob -> Alice: $\{T\}_{Alice}$, g^b mod p

9.1) If NOT consider any attacks...

I. Yes, message 1 shows Alice is Alice. Only Alice can compute $[T]_{Alice}$ using her private key and Bob knows T to verify Alice's signature.

II. No, message 2 does not show Bob is Bob. Everyone can compute $\{T\}_{Alice}$ using Alice's public key, and Alice didn't know g^b mod p beforehand to verify if it's correct.

III. Yes, if Alice and Bob use Ephemeral Diffie Hellman, i.e., they "throw" away a & b after session is done.

9.2) Consider an MiM attack...

I. Yes, MiM will succeed. Alice and Bob didn't know the correct g^b mod p and g^a mod p respectively, so if Trudy can act within time skew, she can be in the middle sending g^t mod p to each of them.

II. Signing the g^a mod p and g^b mod p with T, such as $[T, g^a \bmod p]_{Alice}$ and $[T, g^b \bmod p]_{Bob}$ so Trudy can't change them (recall that digital signature provides integrity).

+Q10: Authentication by public key and symmetric key (L18 P11 & L19 P6)

```
Protocol:
1) Alice -> Bob: "I'm Alice", {S}Bob, E(CONST1, K) // K = h(S)
2) Bob -> Alice: E(CONST2, K)
```

10.1) If NOT consider any attacks...

I. No, message 1 doesn't show Alice is Alice. Everyone can compute $\{S\}_{Bob}$ using Bob's public key, and K is based on S, so whoever picks S, can compute E(CONST1, K) as well.

II. Yes, message 2 shows Bob is Bob. Only Bob can decrypt $\{S\}_{Bob}$ using his private key to get S and then compute K = h(S) to encrypt CONST2 using K. Alice knows K to verify if E(CONST2, K) is correct.

III. Yes, K is exchanged securely. S is encrypted in the 1st message and it is not in the 2nd message. Knowing the ciphertext won't reveal the key used.

10.2) Consider a passive attack (Trudy observes the messages)...

I. Yes, Trudy can find K by brute-force since a 4-digit number only has 10^4 possibilities.

II. No, since a 256-bit key has 2^{256} possibilities, it will take too long to do a brute-force.

Week 11 & 12 – Real-Life & Network Protocols (Q11 - Q20)

Q11: Data transfer over network (L23 P10 – 11)

Down-Up-Down-Up: From sender's application layer, down to the physical layer, then up to the network layer, after that, down to the physical layer again and finally up to the destination's application layer.

When data goes down, each layer adds a header; when data goes up, each layer removes the header.

Q12: Network Protocols (L23)

You can pick any network protocol we covered, for example:

I. HTTP (Hyper Text Transfer Protocol)

II. Application layer

III. Used when browsing a website

+Q13: SSH (L21 P9 – 11)

See the SSH protocol in the lecture notes/question.

13.1) If NOT consider any attacks...

I. Yes, only Bob can compute S_B using his private key, and Alice knows H to verify.

II. Yes, only Alice can compute S_A using her private key, and Bob knows H to verify.

13.2) Change certificate to password

I. No, encryption is not necessary if using certificate since the certificate & signature can be public.

II. Yes, encryption is necessary if using password since the password should be kept secret.

III. No, Trudy can't. The MiM attack will result in different H's for Alice & Bob since H is based on the Diffie-Hellman values and the result shared key. That is, Alice will get H_A, but Bob will get and sign H_B. Therefore, Alice will refuse to authenticate Bob after she checked the 4th message, and she will not send the 5th message. As a result, Trudy cannot get the password in the 5th message.

+Q14: SSL (L21 P13 – 15)

See the SSL protocol in the lecture notes/question.

14.1) If not consider any attacks...

I. No, everyone can compute $\{S\}_{Bob}$ using Bob's public key, and K is based on S, so whoever picks S, can compute E(h(msgs, CLNT, K), K) as well.

II. Yes, only Bob can decrypt $\{S\}_{Bob}$ using his private key to get S and then get K = h(S, R_A, R_B) to compute h(msgs, SRVR, K). And Alice knows K (and everything else) to verify if the response is correct.

III. 1) Encryption in the 3rd message is not necessary, since a secure hash is enough to hide K.
2) R_A and R_B are not necessary since S already serves as a nonce.

14.2) Consider MiM attack on SSL...

I. MiM attack would fail since there's no way for Trudy to fake a certificate.

II. Alice's browser will warn her that the certificate is not valid, but Alice can choose to ignore the warning and continue...then the attack can succeed.

+Q15: Kerberos (L21 P17 – 21)

See the Kerberos protocol in the lecture notes/question.

15.1) Consider the "getting TGT (Kerberized login)" step.

I. TGT is sent to the user so KDC can remain stateless. KDC doesn't need to maintain a database for users and their session keys, since the user will present his/her TGT when getting regular tickets. KDC can decrypt the TGT and get all the information needed.

II. Pros: Security is transparent to Alice; Cons: KDC must be secure – it's trusted!

15.2) Consider the "requesting tickets" step.

I. KDC distributes the shared session keys to reduce the number of shared keys. Everyone is sharing a key with KDC, so N shared keys are needed for N users. If sharing Keys directly between users, need order of N^2 shared keys for N users.

II. Yes, since her name is encrypted in TGT, which can only be decrypted by KDC.

III. Yes, since her name is encrypted in ticket, and only Bob can decrypt it.

IV. Ticket to Bob is sent to Alice, then Alice forwards it to Bob, so Bob can remain stateless. Otherwise, Bob needs to store the ticket and wait for Alice to contact him.

!Q16: Pick all true statements about IPSec (L22 P5 & P8 – 10)

See quiz answers.

Q17: IKE Phase 1 (L22 P12)

No, separating public key encryption and signature as two options is not over-engineering - it is necessary.

Everyone always knows his/her private key but may not initially know other side's public key. So public key signature option is more efficient, since people can start the protocol with their private key first, then search for other's public key simultaneously.

(Next question starts on the next page)

Q18: Plausible Deniability (L22 P14 – 15)

```
Protocol:
1) Alice -> Bob: "I'm Alice", {RA}Bob
2) Bob -> Alice: {RA, RB}Alice
3) Alice -> Bob: E(RB, K) // K = h(RA, RB)
```

I. Yes, Alice is authenticated. Message 3 shows Alice is Alice since only Alice can decrypt $\{R_A, R_B\}_{Alice}$ to get R_B using her private key, and also compute K to encrypt R_B using K. And Bob knows R_B & K to verify if $E(R_B, K)$ is correct.

II. Yes, message 2 shows Bob is Bob, since only Bob can decrypt $\{R_A\}_{Bob}$ to get R_A using his private key and Alice knew R_A to verify.

III. It can provide plausible deniability. Everything in this protocol is a "public" operation: everyone can pick a random R_A and R_B then compute $K = h(R_A, R_B)$; $\{R_A\}_{Bob}$ and $\{R_A, R_B\}_{Alice}$ also can be computed by anyone. That is, Trudy can fake a conversation between Alice & Bob that appears valid to any observer; as a result, Alice & Bob can deny any conversation.

Q19: WEP (L22 P16 – 18)

19.1) Integrity issues:

I. cipher = IP address ⊕ keystream, so Trudy can get keystream = cipher ⊕ IP address. Then she can change the cipher so that cipher = Trudy's IP address ⊕ keystream.

II. CRC can't be used for integrity since it's linear, which means it's easy to solve. CRC only detect unintentional errors, NOT "intelligent" changes. I.e., Trudy can change ciphertext & CRC value so that checksum on plaintext remains valid. A correct algorithm to use for integrity: MAC/HMAC.

19.2) Confidentiality issues:

WEP uses RC4 (a stream cipher) for confidentiality, so using the same key will arise security issues.
The long-term key K is pre-pended with IV to make different keys for different sessions.
But if IV got repeated, then we are using the same key.
Also, with enough IV & ciphertext (both are public), Trudy may be able to find K.

Q20: GSM insecurity (L22 P23 – 24)

Pick any 2 mentioned on the lecture notes (for how to fix, you can be "creative"), for example:

- No encryption from base station to base station controller. To fix this, add encryption there.
- GSM uses weak cryptos to protect data. To fix this, we can use stronger cryptos.
- Base station is not authenticated so there are fake ones. To fix this, also authenticate the base station.
- Replay attacks possible. To fix this, modify the protocol such as signing the nonce, etc.